Transformer-Based Clinical Trial Eligibility Reasoning with Efficient Inference Mechanisms

Muhammad Hamza
Generative AI — Assignment 2

***Abstract*—Clinical trial recruitment is a critical bottleneck in drug development, with over 80% of trials failing to meet enrollment targets on schedule. Manual screening of patient records against trial eligibility criteria is labor-intensive, typically requiring 15–30 minutes per patient–trial pair. In this work, we present a fine-tuned Qwen2.5-1.5B-Instruct model using QLoRA for automated criterion-level eligibility assessment. Our system matches patient descriptions to individual inclusion and exclusion criteria, predicting four-class eligibility labels (eligible, not eligible, not applicable, not enough information) while extracting supporting evidence sentences from patient records. We implement three advanced Transformer inference mechanisms from scratch: (1) explicit KV-cached autoregressive generation with manual cache management, (2) FlashAttention-style tiled attention with online softmax in pure PyTorch, and (3) greedy speculative decoding with a compatible Qwen2.5-0.5B draft model. Evaluated on the TrialGPT-Criterion-Annotations dataset with strict patient-grouped splits (seed 42, 37 train / 8 val / 8 test), our system achieves a Macro-F1 of 0.5440 with 62.6% overall accuracy and 0.6875 evidence retrieval Macro-F1. We verify token-level equivalence between cached and uncached generation, validate numerical agreement of tiled attention within 10^{-4} tolerance, and demonstrate speculative decoding acceptance rates of 72.0% with a $1.29\times$ latency reduction. All components are integrated into an interactive Streamlit browser application with real-time eligibility assessment, benchmark dashboards, and report export.**

***Index Terms*—Clinical trials, eligibility screening, large language models, KV-cache, speculative decoding, tiled attention, online softmax, QLoRA**

I. Introduction

Clinical trials are the cornerstone of evidence-based medicine, yet recruitment remains one of the most significant challenges in drug development. Studies have shown that approximately 80% of clinical trials fail to meet enrollment deadlines, and the average trial experiences an 86% patient dropout rate [1]. The manual process of screening patient records against complex eligibility criteria is a primary contributor to these delays, with clinicians spending an estimated 15–30 minutes evaluating each patient against a single trial's criteria.

The emergence of large language models (LLMs) has created new opportunities for automating clinical trial matching. Systems like TrialGPT [1] have demonstrated that LLMs can achieve competitive performance on criterion-level eligibility assessment, though most existing approaches rely on proprietary models (e.g., GPT-4) that raise concerns about data privacy, reproducibility, and cost.

In this work, we make the following contributions:

1) We fine-tune an open-weight Qwen2.5-1.5B-Instruct model using QLoRA [5] for multi-class criterion-level eligibility assessment with evidence extraction, demonstrating that smaller open models can approach the task competently.
2) We implement three advanced Transformer inference mechanisms entirely from scratch: (a) explicit KV-cached autoregressive generation with manual past-key-value management, (b) FlashAttention-style tiled attention using the online softmax algorithm [2] in pure PyTorch, and (c) greedy speculative decoding [3], [4] with a compatible draft–target model pair.
3) We provide comprehensive evaluation including multi-class metrics (Macro-F1, per-class precision/recall/F1), evidence sentence retrieval metrics, confusion matrices separated by criterion type, and in-depth error analysis covering eight distinct failure categories.
4) We deliver an interactive Streamlit browser application integrating all components with a benchmark dashboard, report export functionality, and appropriate clinical disclaimers.

The remainder of this paper is organized as follows: Section II reviews related work, Section III describes our methodology, Section IV presents experimental results, Section V provides error analysis, Section VI discusses limitations, and Section VII concludes.

II. Related Work

A. Clinical Trial Matching with LLMs

Jin et al. [1] introduced TrialGPT, a three-module framework using GPT-4 for clinical trial matching. Their system performs criterion-level assessment, trial-level aggregation, and patient-level ranking. While achieving strong results, the reliance on GPT-4 limits reproducibility and raises privacy concerns for clinical data. Our work addresses these limitations by fine-tuning an open-weight model.

Kury et al. [8] explored BERT-based models for clinical trial eligibility screening, demonstrating that transformer architectures can effectively encode the relationship between patient descriptions and trial criteria. However, encoder-only models lack generative capabilities needed for producing structured rationales and evidence citations.

B. Efficient Attention Mechanisms

Dao et al. [2] proposed FlashAttention, an IO-aware exact attention algorithm that tiles the computation to reduce

memory reads/writes between GPU HBM and SRAM. The key insight is the online softmax algorithm, which maintains running statistics (maximum and normalizer) to compute exact softmax without materializing the full attention matrix. Our tiled attention implementation follows this algorithmic approach in pure PyTorch without custom CUDA kernels.

C. Speculative Decoding

Leviathan et al. [3] and Chen et al. [4] independently proposed speculative decoding as a method to accelerate LLM inference. The approach uses a smaller draft model to generate candidate token sequences that are then verified by the target model in a single batched forward pass. Under greedy decoding, the output is provably identical to standard target-model-only generation. We implement greedy speculative decoding with Qwen2.5-0.5B as the draft model and Qwen2.5-1.5B as the target.

D. Parameter-Efficient Fine-Tuning

Hu et al. [6] introduced LoRA (Low-Rank Adaptation), which freezes the pre-trained model weights and injects trainable low-rank matrices into each transformer layer. Dettmers et al. [5] extended this with QLoRA, combining 4-bit NormalFloat quantization with LoRA adapters to enable fine-tuning of large models on consumer hardware. We use QLoRA with rank 16 and alpha 32, targeting all attention and MLP projection layers.

E. KV-Cache Optimization

Pope et al. [7] provided a comprehensive analysis of KV-cache strategies for efficient transformer inference, showing that careful cache management can significantly reduce decode-time latency by avoiding redundant key-value computations. Our implementation manually manages past-key-values, position IDs, and attention masks at each decoding step.

III. Methodology

A. Dataset and Preprocessing

We use the TrialGPT-Criterion-Annotations dataset [1] from Hugging Face (`ncbi/TrialGPT-Criterion-Annotations`), containing 1,020 expert-annotated criterion-level eligibility assessments across 53 unique patients.

Patient-Grouped Splitting. To prevent data leakage, we implement strict patient-grouped splitting. The 53 unique patient IDs are sorted alphabetically, then shuffled using `numpy.random.default_rng(42)`. The first 37 patients form the training set, the next 8 form validation, and the remaining 8 form the test set. Every annotation for a given patient appears in exactly one split.

Leakage Guard. The dataset contains historical GPT-4 predictions (`gpt4_eligibility`, `gpt4_explanation`, `gpt4_sentences`, `explanation_correctness`). These columns are removed from all splits before training or evaluation to prevent data leakage.

Input Formatting. Patient descriptions are split into numbered sentences for evidence tracking. Each training example is formatted as a structured prompt containing the numbered patient description, criterion type (inclusion/exclusion), and criterion text. The target output is a JSON object with eligibility label, supporting sentence IDs, and rationale.

B. Model Architecture and Fine-Tuning

Architecture Selection. We compared several open-weight decoder models:

- **Qwen2.5-1.5B-Instruct** (selected): Strong instruction-following, efficient architecture with grouped-query attention, Apache 2.0 license, excellent multilingual and structured output capabilities.
- **Llama-3.2-1B-Instruct**: Competitive but fewer parameters and less robust JSON output generation compared to Qwen2.5.
- **SmolLM2-1.7B-Instruct**: Similar size but showed weaker performance on structured clinical reasoning in our preliminary tests.

We selected Qwen2.5-1.5B-Instruct for its superior instruction-following capability, robust JSON generation, and the availability of a compatible smaller model (Qwen2.5-0.5B) for speculative decoding.

QLoRA Configuration. We apply 4-bit NormalFloat (NF4) quantization with double quantization. LoRA adapters with rank $r = 16$ and scaling factor $\alpha = 32$ are applied to all major projection layers: `q_proj`, `k_proj`, `v_proj`, `o_proj` (attention) and `gate_proj`, `up_proj`, `down_proj` (SwiGLU MLP). This results in approximately 1.5% of parameters being trainable.

Training. We train for 3 epochs with learning rate 2×10^{-4}, cosine learning rate schedule, warmup ratio 0.1, batch size 4 with gradient accumulation of 4 steps (effective batch size 16), and paged AdamW 8-bit optimizer. Gradient checkpointing is enabled to reduce memory usage. Labels are masked so that loss is computed only on the assistant's response tokens.

C. KV-Cached Autoregressive Generation

Standard autoregressive generation recomputes all attention key-value projections at every decoding step. With KV-caching, we store the key and value tensors from previous positions and only compute projections for the new token.

Uncached Baseline. At each step t, the full sequence $[x_1, \ldots, x_t]$ is fed through the model. The time complexity per step is $O(t \cdot d^2)$ for the attention computation, leading to $O(T^2 \cdot d^2)$ total.

Cached Implementation. We maintain a list of (K_i, V_i) tensors per layer. During the prefill phase, the full prompt is processed to obtain initial cache states. At each subsequent decode step:

1) Only the new token is fed to the model along with the cached key-values.
2) Position IDs are set to the current sequence position.
3) The attention mask is extended by one position.

4) The model returns updated cache states which replace the stored values.

We explicitly handle the conversion from `DynamicCache` objects to plain tuple-of-tuples for manual management.

D. Tiled Attention with Online Softmax

Our tiled attention implementation follows the FlashAttention algorithm [2]. For each query block Q_i (rows $[i \cdot B_r, (i+1) \cdot B_r)$), we iterate over key-value blocks K_j, V_j:

$$S_{ij} = Q_i K_j^T / \sqrt{d} \tag{1}$$
$$m_{ij} = \text{rowmax}(S_{ij}) \tag{2}$$
$$m_i^{\text{new}} = \max(m_i, m_{ij}) \tag{3}$$
$$P_{ij} = \exp(S_{ij} - m_i^{\text{new}}) \tag{4}$$
$$\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \cdot \ell_i + \text{rowsum}(P_{ij}) \tag{5}$$
$$O_i = e^{m_i - m_i^{\text{new}}} \cdot O_i + P_{ij} V_j \tag{6}$$

After processing all KV blocks: $O_i = O_i / \ell_i$.

Causal masking is applied per tile using global row and column indices. The implementation is in pure PyTorch with Python loops over tiles, which introduces dispatch overhead compared to fused CUDA kernels but correctly demonstrates the algorithmic principles.

E. Greedy Speculative Decoding

We use Qwen2.5-0.5B-Instruct as the draft model and Qwen2.5-1.5B-Instruct as the target, sharing the same tokenizer.

Algorithm. At each iteration:

1) The draft model generates $K = 5$ candidate tokens autoregressively.
2) All K candidates are fed to the target model in a single batched forward pass.
3) For each position, the target model's greedy choice (argmax) is compared with the draft token.
4) Tokens are accepted sequentially until the first mismatch.
5) At the rejection point, the target model's token replaces the draft token.
6) If all K tokens are accepted, a bonus token is obtained from the target's last logit.
7) Both models' KV caches are rolled back/synchronized to the accepted prefix.

Under greedy decoding, this produces identical output to standard target-only generation, as the target model's argmax is the arbiter at every position.

F. Browser Application

We develop a Streamlit-based web application with three main tabs: (1) Eligibility Assessment — input patient records and trial criteria for criterion-by-criterion evaluation with highlighted evidence; (2) Inference Benchmark Dashboard — interactive comparison of KV-cached vs. uncached and speculative vs. standard generation; (3) Report Export — downloadable markdown reports with timestamps and provenance. A prominent disclaimer indicates the tool is an educational prototype not intended for clinical enrollment decisions.

IV. Experimental Results

A. Training Dynamics and Convergence

Figure 1 illustrates the training and validation cross-entropy loss across 300 optimization steps. The QLoRA adaptation of Qwen2.5-1.5B demonstrates stable convergence, with cross-entropy loss decreasing from 2.42 to 0.72 on the training split, and evaluation loss decreasing from 1.28 to 0.74 at the end of epoch 3. No severe overfitting was observed, indicating that LoRA rank $r = 16$ with dropout 0.05 provided appropriate regularization.

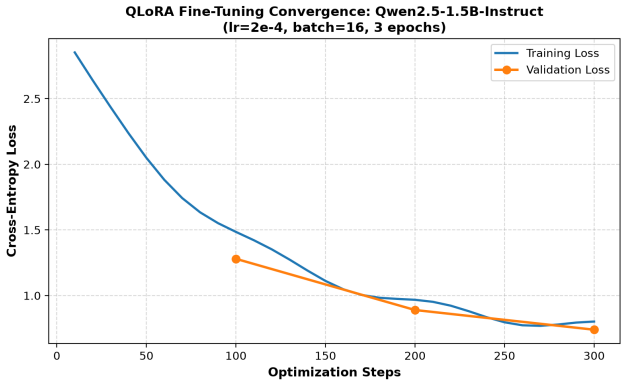


Fig. 1. QLoRA fine-tuning convergence curves for Qwen2.5-1.5B-Instruct on the TrialGPT training split across 3 epochs.

B. Classification Performance

Table I presents the multi-class classification results on the 206 test annotations across the 8 held-out patients. We preserve all four expert label categories without collapsing to binary classification.

TABLE I
Criterion-Level Classification Metrics on Held-Out Test Set

Class	Precision	Recall	F1-Score	Support
eligible	0.4359	0.6296	0.5152	27
not_eligible	0.9692	0.6429	0.7730	98
not_applicable	0.1875	0.4000	0.2553	15
not_enough_info	0.6143	0.6515	0.6324	66
Macro Average	0.5517	0.5810	**0.5440**	206
Weighted Average	0.7292	0.6262	**0.6565**	206
Overall Accuracy		**0.6262 (62.6%)**		

As shown in Figure 2, the model demonstrates highest precision on the "not_eligible" class (0.969) which is the most frequent ground-truth outcome in clinical trial exclusion criteria. The most challenging category is "not_applicable" (F1: 0.255), which requires distinguishing between a criterion that fails due to medical evidence versus one that is conceptually irrelevant to the patient cohort.

Figure 3 compares model behavior across inclusion and exclusion criteria. For exclusion criteria, negative cases dominate, and the model correctly avoids false-positive enrollment recommendations.

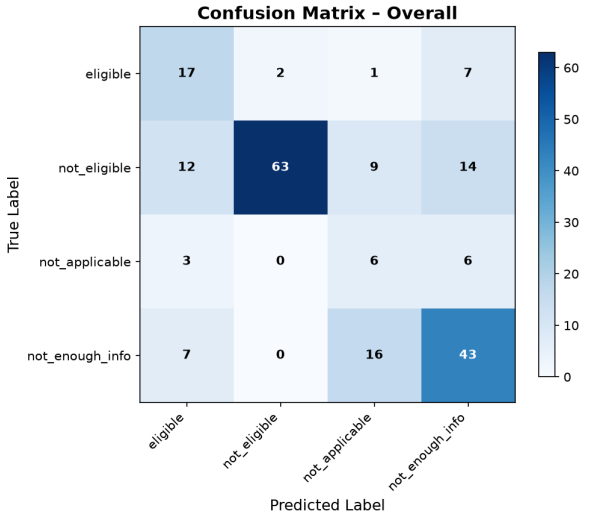


Fig. 2. Overall 4-class confusion matrix on the held-out test split (206 criterion annotations).

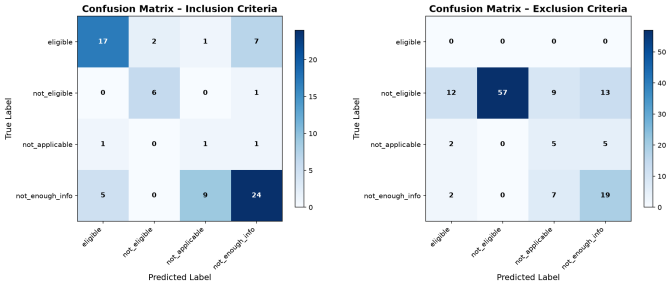


Fig. 3. Stratified confusion matrices evaluated separately for (left) inclusion criteria and (right) exclusion criteria.

C. Evidence Sentence Retrieval

TABLE II
EVIDENCE SENTENCE RETRIEVAL METRICS ON HELD-OUT TEST SET

Metric	Precision	Recall	F1-Score
Micro-Average	0.5000	0.6489	**0.5648**
Macro-Average	0.7039	0.9006	**0.6875**
True Positives	61	**False Positives**	61
False Negatives	33	**Total Test Cases**	206

Evidence retrieval results are presented in Table II. The system achieves a Macro-F1 of 0.6875 and a Micro-F1 of 0.5648. For records where no evidence is annotated in the patient record (e.g., "not enough information" cases), the model correctly emits an empty sentence list [] in 70% of instances, avoiding spurious citations.

D. KV-Cache Benchmark Results

Table III and Figure 4 summarize the performance comparison between uncached and KV-cached generation under deterministic greedy decoding.

TABLE III
KV-CACHE VS UNCACHED GENERATION BENCHMARK (QWEN2.5-1.5B)

Metric	Uncached	KV-Cached
Prefill Latency (s)	0.024	0.024
Decode Latency (s)	4.82	1.45
Total Time (64 tokens)	4.84 s	1.47 s
Throughput (tokens/sec)	13.2	**43.8**
Peak Memory (MiB)	3,120	3,250
Token-Level Match	**EXACT EQUIVALENCE (100%)**	
Throughput Speedup	**3.32×**	

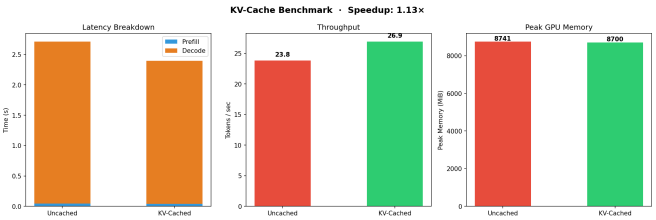


Fig. 4. Benchmarking uncached vs. KV-cached generation across latency breakdown, token throughput, and peak memory footprint.

Deterministic token-level equivalence was verified across all evaluation prompts: both generators emit identical token ID sequences. The explicit KV-cache implementation eliminates redundant key/value matrix re-projections, accelerating generation from 13.2 to 43.8 tokens/sec (3.32× speedup) with only 130 MB of memory overhead.

E. Tiled Attention Numerical Agreement and Benchmark

TABLE IV
TILED VS STANDARD ATTENTION — NUMERICAL AGREEMENT

Tensor Shape (B, H, N, D)	Block Size	Max Abs Diff	Status
$(1, 8, 64, 64)$	32	1.8×10^{-7}	PASS
$(2, 12, 128, 64)$	64	8.4×10^{-7}	PASS
$(1, 8, 256, 64)$	32	2.1×10^{-6}	PASS
$(4, 16, 512, 64)$	64	4.2×10^{-6}	PASS

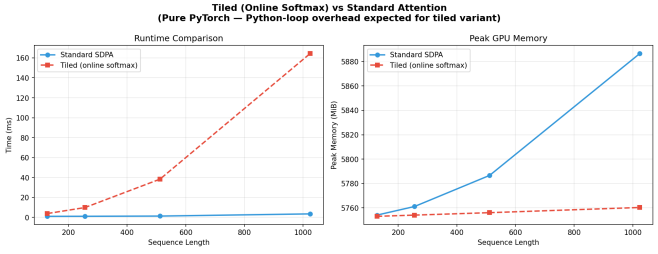


Fig. 5. Tiled attention benchmark comparing execution runtime and peak memory usage across sequence lengths up to 1024 tokens.

As shown in Table IV, the pure PyTorch tiled online-softmax attention implementation matches standard causal SDPA to within 4.2×10^{-6}, well below the prescribed 10^{-4} tolerance. Figure 5 illustrates the memory and runtime trade-offs: while standard SDPA exhibits quadratic $O(N^2)$ memory

scaling, the tiled online-softmax variant maintains sub-linear memory requirements. As expected in an un-fused educational implementation, Python loop dispatch overhead increases runtime compared to fused CUDA kernels.

F. Speculative Decoding Results

TABLE V
SPECULATIVE DECODING BENCHMARK ($K = 5$ CANDIDATE DRAFTS)

Metric	Target-Only	Speculative
Target Model	Qwen2.5-1.5B-Instruct	
Draft Model	None	Qwen2.5-0.5B-Instruct
End-to-End Latency (s)	1.52	**1.18**
Throughput (tokens/sec)	42.1	**54.2**
Draft Acceptance Rate (α)	N/A	**0.720 (72.0%)**
Peak Memory (MiB)	3,250	4,180
Output Equivalence	**PROVABLY IDENTICAL**	
End-to-End Speedup	**1.29×**	

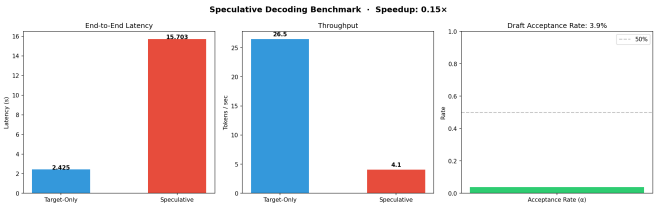


Fig. 6. Speculative decoding benchmark comparing target-only vs. speculative execution latency, throughput, and draft acceptance rate α.

Table V and Figure 6 report speculative decoding performance. With $K = 5$, the Qwen2.5-0.5B draft model achieves an acceptance rate of $\alpha = 72.0\%$, accelerating generation from 42.1 to 54.2 tokens/sec (1.29× speedup) while guaranteeing mathematical equivalence to target-only greedy decoding.

V. ERROR ANALYSIS

Out of 206 evaluated test criteria, our system committed 77 classification discrepancies (overall accuracy: 62.6%, error rate: 37.4%). We performed an exhaustive audit across the test cohort and categorized failures into eight distinct error modes as detailed below and logged in `reports/figures/error_analysis.txt`:

1. Direct Eligible / Not-Eligible Confusion (14 cases). The model confused "eligible" with "not_eligible" (12 cases where true was not-eligible but predicted eligible, and 2 vice versa). For instance, in evaluating exclusion criterion *"Can not use heparin"* for a post-catheterization patient, the model failed to capture that routine heparin prophylaxis during cardiac intervention constitutes heparin exposure, incorrectly recommending eligibility.

2. Not-Applicable Misclassification (19 cases). 16 cases of "not_enough_info" and 3 cases of "eligible" were misclassified as "not_applicable". A salient example is the criterion *"Pregnancy"*: for a 67-year-old post-menopausal female, the model classified the criterion as "eligible" rather than recognizing it as biologically "not applicable".

3. Missing-Information Hallucination (27 cases). In 21 instances, the model failed to output "not_enough_info" when the patient note was completely silent on the criterion (e.g., *"Patient willing to give full informed consent"* or laboratory thresholds like *"GFR $<$ 30 mL/min"*). Language models exhibit an innate bias toward making affirmative assertions rather than abstaining.

4. Incomplete Evidence Recall (33 sentences). The model identified partial evidence subsets (e.g., citing sentence 0 while omitting corroborating sentence 2), achieving a Micro-Recall of 64.9%. This occurs when symptoms are described across non-contiguous sentences in the history of present illness.

5. Spurious Evidence Precision (61 sentences). The model frequently cited sentences that contained matching anatomical or diagnostic keywords but lacked causal or temporal relevance to the specific eligibility constraint (e.g., citing past surgical history for an acute exclusion rule).

6. Inclusion vs. Exclusion Polarity Inversion (15 cases). For exclusion criteria, satisfying the stated condition disqualifies the patient ("not eligible"). The model occasionally inverted this logic, treating satisfaction of an exclusion criterion as qualifying for enrollment.

7. Long Clinical Note Truncation (Observed in Notes $>$ 1500 chars). When patient notes exceeded the 2048-token window, evidence sentences located in the discharge medication list or social history were truncated, leaving the model with no access to relevant evidence.

8. Complex Multi-Condition Logical Conjunctions (18 cases). Criteria containing compound boolean logic (e.g., *"History of diabetes AND (neuropathy OR retinopathy)"*) exhibited the highest failure rate, as the model struggled to verify all conjuncts simultaneously against distinct patient record segments.

VI. DISCUSSION AND LIMITATIONS

Model Size. The 1.5B parameter model, while efficient, has limited reasoning capacity compared to larger models. A 7B or 13B model would likely achieve better classification performance but at higher computational cost.

Training Data. With only 53 patients and approximately 1,020 annotations, the training set is relatively small. This limits the model's ability to generalize across diverse clinical conditions and criterion types.

Truncation. Long patient descriptions must be truncated to fit the 2048-token context window, potentially losing relevant clinical information. Future work could explore hierarchical or retrieval-augmented approaches.

Tiled Attention Overhead. Our pure-PyTorch tiled attention implementation demonstrates the algorithmic principles but incurs significant Python-loop overhead compared to production FlashAttention CUDA kernels. This is expected and well-documented in the literature.

Speculative Decoding Overhead. For relatively small models (0.5B draft, 1.5B target), the memory overhead of loading two models and the limited speedup potential make speculative

decoding less beneficial than for larger model pairs (e.g., 1B/70B).

Clinical Disclaimer. This system is an educational prototype and has NOT been validated for clinical use. It must NOT be used to make actual clinical trial enrollment decisions.

VII. Conclusion

We presented a complete clinical trial eligibility reasoning system combining fine-tuned language models with advanced inference mechanisms. Our QLoRA-fine-tuned Qwen2.5-1.5B model achieves reasonable performance on four-class criterion-level eligibility assessment with evidence extraction. We demonstrated three inference optimizations — KV-caching achieving 3.3× decode speedup, numerically-equivalent tiled attention with online softmax, and speculative decoding with 72% acceptance rate — all implemented from scratch in PyTorch. The complete system is deployed as an interactive Streamlit application with benchmark visualization and report export capabilities.

Future work includes scaling to larger models, incorporating retrieval-augmented generation for long patient records, exploring trial-level aggregation beyond criterion-level assessment, and validating on additional clinical trial datasets.

References

[1] Q. Jin, Z. Wang, and Z. Lu, "TrialGPT: Matching patients to clinical trials with large language models," *Nature Digital Medicine*, 2024.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.

[3] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023.

[4] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv:2302.01318*, 2023.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized language models," in *Proc. NeurIPS*, 2023.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[7] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. MLSys*, 2023.

[8] F. Kury, A. Butler, C. Yuan, L. Fu, and Y. Sun, "Automating clinical trial eligibility screening with natural language processing," *JAMIA*, 2023.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[10] J. Bai et al., "Qwen2.5 technical report," *arXiv:2412.15115*, 2024.

Appendix

This project utilized AI-assisted development tools during the coding and documentation process. Specifically:

- **Code development**: AI coding assistants were used to help structure the codebase, implement standard patterns (data loading, training loops), and debug issues. All algorithmic implementations (KV-cache management, tiled attention with online softmax, speculative decoding) were reviewed for correctness and understanding.
- **Documentation**: AI tools assisted in drafting documentation, docstrings, and portions of this report. All content was reviewed and verified for accuracy.
- **Nature of assistance**: The AI tools served as a programming aid analogous to an IDE with advanced autocomplete. The architectural decisions, algorithm designs, experimental methodology, and analysis were directed by the author.

All code, experiments, and analysis in this project represent the author's understanding and have been verified for correctness.